ERHVERVSAKADEMI KØBENHAVN

Lesson — ER Modeling + DDL/DML (Full Build Session)

From model → tables → data → verification
MariaDB in Docker + DBeaver

Exam Assignment Introduction

Purpose: what the assignment is testing (ER + build + data + verification)
Deliverables: 1× ER diagram (draw.io), SQL script(s): DDL + DML, Short README
Constraints: naming conventions, keys, minimum tables, sample data size
Assessment focus: correctness, clarity, normalization thinking, evidence via SELECTs

Outcomes for Today

- Model a small domain using ER concepts (entities, attributes, identifiers)
- Specify relationships with cardinality + optionality (0..1 / 0..* / 1..*)
- Map ER → tables (including M:N → junction table)
- Create a schema with DDL (CREATE DATABASE/TABLE, data types, PK/FK, ALTER)
- Load and change data with DML (INSERT, UPDATE, DELETE) safely
- Use a repeatable DBeaver workflow: run script → refresh → verify

Session Workflow — High-Level

- 1) ER model: identify entities/attributes/keys
- 2) Relationships: cardinality + optionality
- 3) Map to tables (students/courses/enrollments)
- 4) Build schema with DDL
- 5) Load & edit data with DML
- 6) Verify with SELECTs after each step

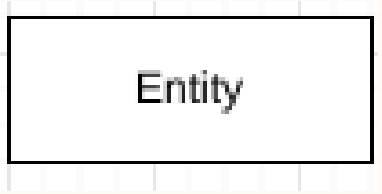
DBeaver Safe Workflow — Do This Every Time

- Open SQL Editor on the correct connection
- Run small chunks (not the whole script blindly)
- Refresh the schema tree after DDL (right-click → Refresh)
- Verify immediately using SELECTs (spot-check, counts, limits)
- If something looks wrong: fix with targeted DDL/DML and re-verify

ER Model Foundations

ER Essentials

- **Entity:**
- A thing we store data about (Student, Course, Enrollment)



- **Attribute:**
- A property of an entity (first_name, start_date, ects)



- **Identifier:**
- Attribute(s) that uniquely identify an entity (primary key concept)

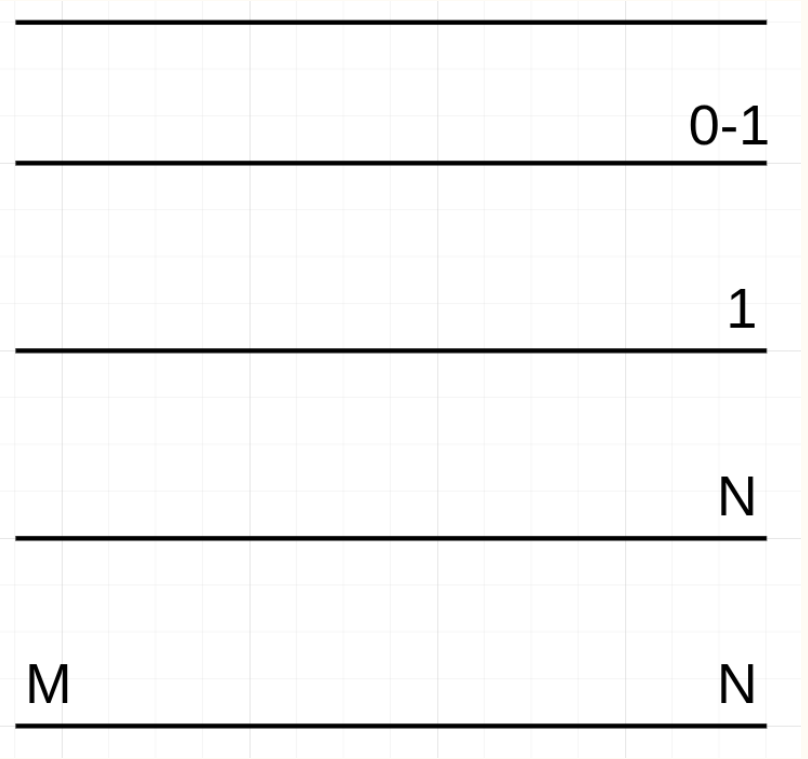
- **Rule of thumb: identifiers must be stable and unique**

Identifiers (Primary Keys): What to Pick?

- **Natural key:**
  - Comes from the real world (email, student_number)
  - Pros: meaningful
  - Cons: can change, privacy, complex
- **Surrogate key:**
  - System-generated id (INT/BIGINT)
  - Pros: stable, simple joins
  - Cons: not meaningful
  - Typical in teaching + many systems

Relationships + Notation

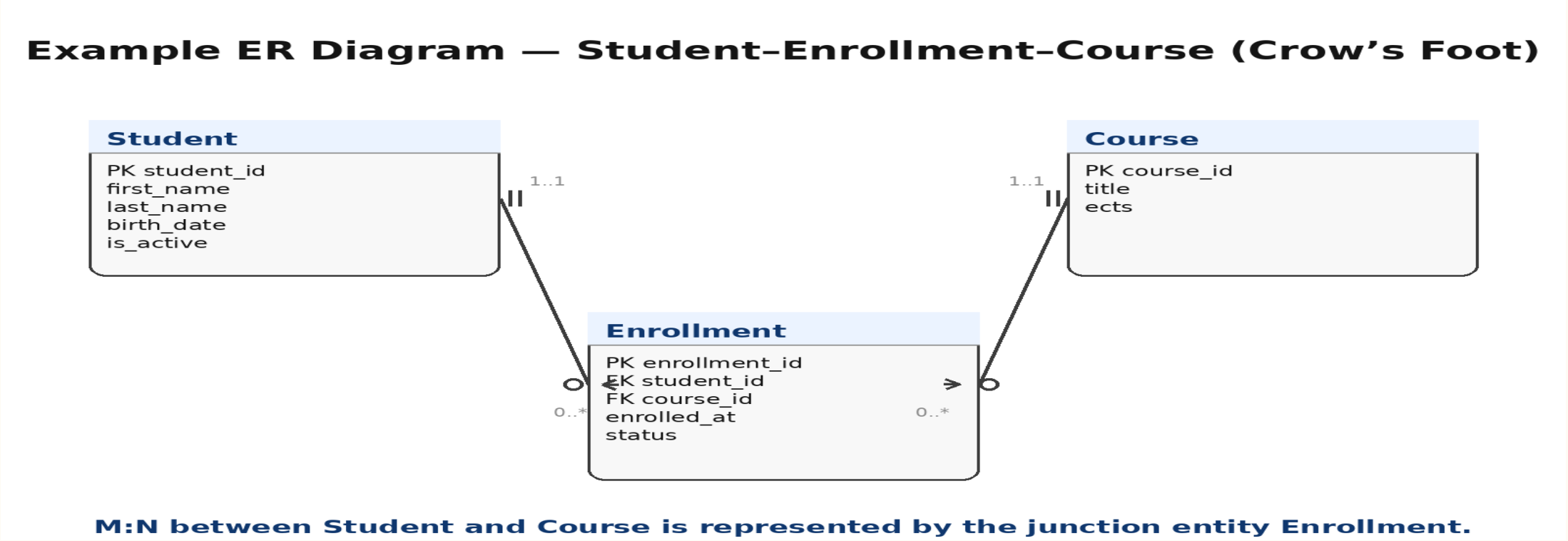
- **Relationship types:**
  - 1:1 — one A relates to one B (less common)
  - 1:M — one A relates to many B (common)
  - M:N — many A relate to many B (requires junction table)



- **Crow's Foot notation symbols:**
  - Entity ——| |—— Entity (exactly one to exactly one)
  - Entity ——o |—— Entity (zero or one to exactly one)
  - Entity ——< |—— Entity (many to exactly one)
  - Entity ——o< |—— Entity (zero or many to exactly one)

- **| = one (mandatory), o = zero (optional), < = many**

Example Domain We Will Model



- **Relationships:**
  - Student (1) ——o< (0..*) Enrollment: A student can have zero or many enrollments
  - Course (1) ——o< (0..*) Enrollment: A course can have zero or many enrollments
  - Enrollment (1) ——| (1..1) Student: Each enrollment belongs to exactly one student
  - Enrollment (1) ——| (1..1) Course: Each enrollment belongs to exactly one course<

Draw.io (diagrams.net) for ER Diagrams

- Create new diagram → ER template (Crow's Foot if available)
- Use singular nouns for entities (Student, Course)
- Put the identifier (PK) at the top of attributes
- Show cardinality + optionality on both ends of each relationship
- Export/share as evidence for the exam deliverable

Mapping ER → Tables — Rules You Must Know

Transformation rules:

Entity → Table




```
CREATE TABLE Student (
  student_id INT PRIMARY KEY,
  first_name VARCHAR(50)
);
```

1:M relationship → Foreign Key on 'many' side

Dept (1) ——< (M) Employee → Employee table gets dept_id as FK

M:N relationship → Junction Table

Student >——< Course → CREATE TABLE Enrollment (
student_id FK, course_id FK, ...
);

Why M:N Becomes a Junction Table

The problem with storing lists in a single row (breaks 1NF):

❌ BAD: Student table with course list

student_id	courses	
1	'Math,Physics,Chemistry'	← violates 1NF!

✓ GOOD: Junction table (Enrollment)

student_id	course_id	enrolled_at
1	101	2024-01-15
1	102	2024-01-15
1	103	2024-01-16

Each enrollment = one row with relationship attributes

ERHVERVSAKADEMI KØBENHAVN

Normal Forms

Normal Forms — 30,000 ft View

- **1NF:**
- Atomic values; no repeating groups; no 'lists in a column'

- **2NF:**
- No partial dependency on part of a composite key (only relevant with composite keys)

- **3NF:**
- No transitive dependencies (non-key depends on non-key)

- **Use as a quick sanity check to reduce duplication and anomalies**

DDL (Build the Database)

DDL Basics: Database Lifecycle

- CREATE DATABASE school;
- USE school; (or select schema in DBeaver)
- DROP DATABASE school; (dangerous: deletes everything)

DDL Basics: CREATE TABLE Essentials

- • Choose columns + data types
- Define PRIMARY KEY
- Add NOT NULL where appropriate
- Add FOREIGN KEY constraints when ready
- Evolve schema with ALTER TABLE

Common MariaDB Data Types — Practical Set

- INT / BIGINT — ids, counters
- VARCHAR(n) — names, titles
- DATE — birthdays, dates
- DATETIME — timestamps
- DECIMAL(p,s) — money/grades
- BOOLEAN — flags (often TINYINT(1))
- TEXT — long notes (sparingly)
- ENUM — limited values (cautiously)

Constraints You Should Use Today

- PRIMARY KEY — uniqueness + identity
- NOT NULL — mandatory data
- UNIQUE — enforce uniqueness (e.g., one enrollment per student per course)
- DEFAULT — safe defaults (e.g., is_active default 1)
- FOREIGN KEY — relationship integrity

ALTER TABLE — How Schemas Evolve

- ADD COLUMN (new attributes)
- MODIFY COLUMN (change type/size)
- ADD CONSTRAINT (e.g., FK/UNIQUE after initial load)

- **Do small ALTERs and verify schema after each change**

DML (Load and Change Data)

INSERT Patterns You Must Know

- INSERT with column list (recommended)
- INSERT without column list (fragile if schema changes)
- Insert multiple rows in one statement
- Verify using SELECT after insert

UPDATE Safely

- Always use WHERE
- Preview first: SELECT with the same WHERE
- Run UPDATE
- Verify again with SELECT

DELETE Safely

- Always use WHERE
- Preview first: SELECT rows to be deleted
- Run DELETE
- Verify row counts
- Foreign keys may block deletes depending on design

Practice Bursts — Repeatable

Practice 1 — ER Identification

From a short case text:

- List entities and attributes
- Choose identifiers (PK) for each entity
- Write 2-3 relationship sentences

Practice 2 — Draw ER in draw.io

- Draw Student–Enrollment–Course
- Add PK attributes
- Add cardinality + optionality
- Partner check: does it allow the scenario?

Practice 3 — Map ER → Tables

- List tables + columns
- Mark PKs
- Identify FKs (enrollments.student_id, enrollments.course_id)
- Decide: surrogate PK vs composite PK in Enrollment

Practice 4 — Write DDL

- Create database + tables
- Add PK, NOT NULL, DEFAULT
- Add UNIQUE rule: one enrollment per student per course
- Add FK constraints and verify in DBeaver

Practice 5 — Load & Edit Data

- Insert 5 students, 5 courses, 10 enrollments
- Update one enrollment status
- Delete one enrollment (with WHERE)
- Verify with SELECT * and COUNT(*) after each step

ERHVERVSAKADEMI KØBENHAVN

Wrap-up Checklist

✓ I can explain ER basics and choose identifiers
✓ I can model M:N using a junction table
✓ I can create tables with correct data types and PK/FK
✓ I can insert/update/delete data safely
✓ I can prove correctness with verification SELECTs